

# Local, or global?

What LIME really computes, and which half of its answer you can trust

Lecture

**William Bendinelli**

**ICMC-USP**

Topics in Artificial Intelligence

**ACT 1 Build**

The method, taught properly: what LIME optimizes and the six steps that implement it.

**ACT 2 Break**

Three measurements on our own run that undo part of what Act 1 just taught.

**ACT 3 Rebuild**

What survives the breaking, and the rules that follow for using it.

**→ One dataset throughout**

569 breast-cancer patients, one RandomForest, one patient explained.

---

ACT 1

# Build

This is LIME as its authors describe it, and as the package builds it. We take it at face value until Act 2.

## Four words, four commitments.

The first word is the one people skip. LIME explains one prediction at a time, never the whole model. And the explanation can use features the model never saw: train on PCA components, explain in the original survey questions.

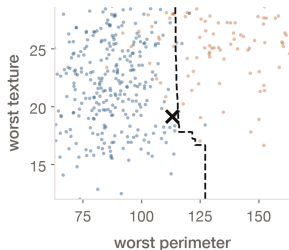
**LOCAL****faithful near one point****INTERPRETABLE****readable by a human****MODEL-AGNOSTIC****any model, any data****EXPLANATIONS****input → prediction, legible**

## One equation, three demands.

$$\xi(x) = \arg \min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

- 01 Faithful to the black box —  $\mathcal{L}$  against  $f$ , a RandomForest of 300 trees
- 02 Faithful *here* —  $\pi_x = \sqrt{\exp(-d^2/\nu^2)}$ , width  $\nu = 0.75\sqrt{30} \approx 4.11$
- 03 Readable —  $\Omega$ : 8 of 30 features, and  $g$  is a Ridge with  $\alpha = 1$

## THE PICTURE



● benign ● malignant × patient #67  
Dashed line: the forest's own decision boundary in this slice.

## THE CODE

```
model = RandomForestClassifier(  
    n_estimators=300,  
    min_samples_leaf=3,  
) .fit(X_train, y_train)  
  
# the only thing we ever  
# ask of f, all deck long  
p = model.predict_proba(X)[: , 1]
```

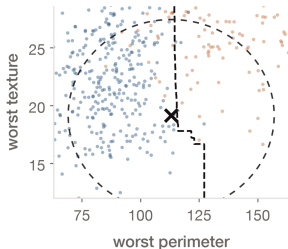
## THE IDEA

## Explain one prediction, not the model.

We can call the model, but not look inside it. Patient #67 is used throughout. She sits on the boundary, so the geometry stays visible. The axes are her strongest and third-strongest features.

**SO FAR — a model, and one row.**

## THE PICTURE



● benign ● malignant × patient #67  
Dashed ring: the kernel's 0.95-weight contour, at  $1.32\sigma$ .

## THE CODE

```
# one width per feature,  
# each on its own scale  
mu = X_train.mean(axis=0)  
sigma = X_train.std(axis=0)  
  
# LIME's default kernel width  
# lime_tabular.py:243  
nu = 0.75 * np.sqrt(30)
```

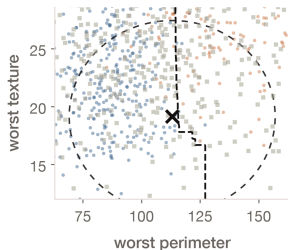
## THE IDEA

**Locality must be defined before it means anything.**

We pick a distance and a width. The data does not pick them for us. Only one kernel contour fits this frame: the one where the weight is 0.95. The half-weight contour is far outside it, and Act 2 shows how far.

**SO FAR — + a definition of "near".**

## THE PICTURE



● benign ● malignant × patient #67

■ synthetic neighbor

All 30 features resampled, not just these two.

## THE CODE

```
# sample_around_instance=False
# is the default, so the cloud
# is centred on the DATASET,
# not on the patient
Z = mu + sigma * rng.normal(
    0, 1, size=(5000, 30))

Zs = (Z - mu) / sigma
```

## THE IDEA

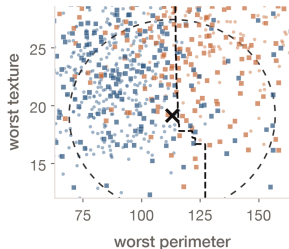
**If you cannot look inside,  
build data to probe with.**

This is where the assumptions enter.  
Each of the 30 features is drawn on its  
own, ignoring how features move  
together. And the cloud is centred on the  
dataset, not on the patient.

**SO FAR — + 5,000 probe rows.**



## THE PICTURE



● benign ● malignant × patient #67  
■ pred. benign ■ pred. malignant  
Square colour is the model's prediction, not a true label.

## THE CODE

```
# the one moment the real  
# model is queried at all –  
# 5,000 rows of 30 numbers  
fz = model.predict_proba(Z)[: , 1]  
  
# note: raw Z, not Zs. The model  
# was trained on raw units.
```

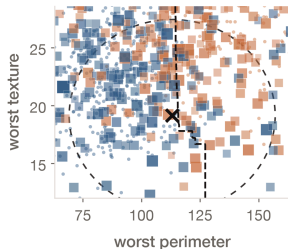
## THE IDEA

**The black box becomes its own teacher.**

Now LIME has a labelled dataset to learn from. Notice that the squares do not split along the drawn boundary: 68%, against a 53% baseline. They cannot. Each one carries random values on the other 28 features.

**SO FAR — + the model's answer on each.**

## THE PICTURE



● benign ● malignant × patient #67  
■ pred. benign ■ pred. malignant  
Square size and opacity encode the proximity weight  $\pi$ .

## THE CODE

```
# distance over all 30  
# STANDARDIZED features,  
# not the two we plot  
d = np.linalg.norm(  
    Zs - x_scaled, axis=1)  
  
w = np.sqrt(  
    np.exp(-d**2 / nu**2))
```

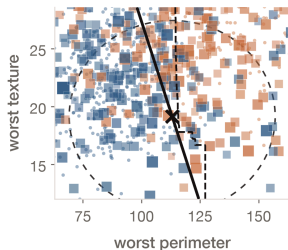
## THE IDEA

**This is the only step that makes the fit local.**

Everything before this was global. The kernel is supposed to pull the fit towards the patient. Watch how little the weights actually vary.

**SO FAR — + a weight per row.**

## THE PICTURE



● benign ● malignant × patient #67  
■ pred. benign ■ pred. malignant  
Solid line: the local fit, drawn through the patient.

## THE CODE

```
# stage 1 - pick 8 of 30,  
# itself weighted by w  
aux = Ridge(alpha=0.01).fit(  
    Zs, fz, sample_weight=w)  
top8 = np.argsort(  
    -abs(aux.coef_ * x_scaled))[:8]  
  
# stage 2 - the explanation  
g = Ridge(alpha=1).fit(  
    Zs[:, top8], fz,  
    sample_weight=w)
```

## THE IDEA

**The fitted model is the explanation.**

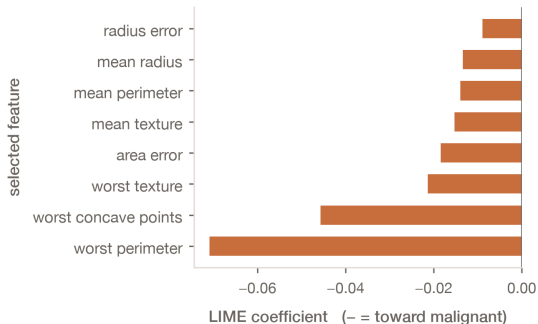
Two weighted stages: pick 8 features of 30, then fit. The coefficients are what you report. The line here is that same fit seen in the plane, drawn through the patient instead of at  $P = 0.5$ .

**DONE — 8 coefficients, and  $R^2 = 0.36$ .**

## What the library actually hands you.

- 01 What you actually read is a ranked list, never the geometry we just drew
- 02 Here it is unanimous: all eight selected measurements say malignant
- 03 But the model says benign. So the decision rests on the 22 features LIME left out

All eight selected features point the same way



---

ACT 2

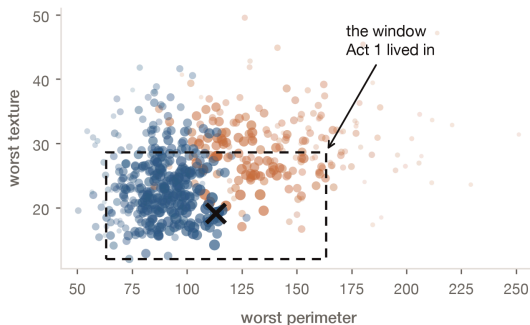
# Break

Three claims from Act 1, put to the test. Two are already in the literature, and we check them here. The third is ours, and it does not survive a second dataset.

## The "local" fit is barely local.

- 01 Garreau & von Luxburg (2020) proved it. At the default width the kernel "becomes redundant: it is equivalent to give weight 1 to every perturbed sample"
- 02 Delete the kernel entirely and  $g(x)$  moves by 0.002. All 8 features stay. The same holds across four datasets and three model classes
- 03 Its half-weight contour sits at  $4.84\sigma$ , so 42% of all 569 patients carry a weight above 0.5
- 04 And nobody knows how to choose the width. A narrow one drives every coefficient to zero, and the authors call the search for a principled rule "still open"

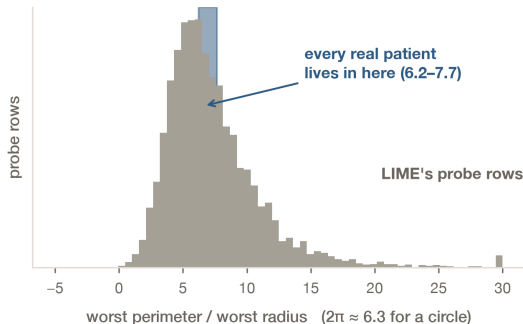
Dot size is each patient's actual kernel weight



## It explains data that cannot exist.

- 01 76% of the synthetic neighbours carry a negative measurement. A tumour cannot have one
- 02 Perimeter divided by radius runs from 1.8 to 21.5 in the cloud. In real patients it stays between 6.2 and 7.7
- 03 Those fake rows are easy to spot, and Slack et al. (2020) used that to hide a racist classifier from LIME
- 04 Molnar already lists this limitation. We are measuring it, not discovering it

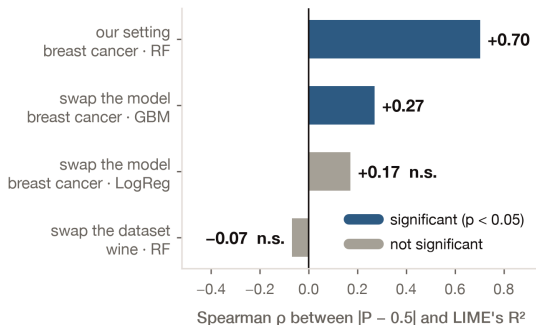
Three quarters of the probe rows are impossible patients



## And one we measured that does not hold up.

- 01 On this dataset with this model,  $R^2$  tracks how confident the prediction was:  $\rho = +0.70$
- 02 Swap the model and it weakens. Swap the dataset for *wine* — 13 chemical measurements, nothing to do with cancer — and it disappears
- 03 A published study finds *no* consistent pattern at all, and one case running the other way (Velmurugan et al., 2020)

It survives only the setting we happened to study first





---

ACT 3

# Rebuild

None of this says throw LIME away. It says: know which half of the answer to read. And that half can be measured.

**A linear fit gives you two things. Only one of them holds — and only at one scale.**

DISCARD

The *level*:  $g(x) = 0.497$  while  $f(x) = 0.581$ .

---

KEEP

The *direction, at the scale LIME sampled*: the leading coefficient carries the right sign for 85% of patients.

MEASURED AT ONE SD, LIME'S OWN STEP · COSINE  
0.81 · 143 PATIENTS

**01 Read the direction, not the level**

Use the sign of the top coefficient — right 85% of the time. Never the surrogate's own predicted probability.

**02 Never rank explanations by  $R^2$** 

You cannot know what it is tracking in your own setting. Report it as a caveat, not a score.

**03 Check  $g(x)$  against  $f(x)$** 

One line of code. If they disagree, the level is worthless. Usually they do: the median gap is 0.222.

**04 Re-run before you believe a ranking**

The top feature holds. The tail moves. Trust only what survives a new seed.

# Thanks!



[github.com/wbendinelli/interpretable-ml-lectures](https://github.com/wbendinelli/interpretable-ml-lectures)

## REFERENCES

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *Proceedings of KDD '16*, 1135–1144. doi:10.1145/2939672.2939778

Garreau, D., & von Luxburg, U. (2020). Explaining the Explainer: A First Theoretical Analysis of LIME. *Proceedings of AISTATS 2020*, PMLR 108, 1287–1296

Garreau, D., & von Luxburg, U. (2020). Looking Deeper into Tabular LIME. arXiv:2008.11092

Slack, D., Hilgard, S., Jia, E., Singh, S., & Lakkaraju, H. (2020). Fooling LIME and SHAP: Adversarial Attacks on Post hoc Explanation Methods. *Proceedings of AIES '20*, 180–186. doi:10.1145/3375627.3375830

Velmurugan, M., Ouyang, C., Moreira, C., & Sindhgatta, R. (2020). Evaluating Explainable Methods for Predictive Process Analytics: A Functionally-Grounded Approach. arXiv:2012.04218

Alvarez-Melis, D., & Jaakkola, T. S. (2018). On the Robustness of Interpretability Methods. *ICML Workshop on Human Interpretability in Machine Learning*. arXiv:1806.08049

Molnar, C. *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*, LIME chapter. christophm.github.io/interpretable-ml-book

Street, W. N., Wolberg, W. H., & Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. *IS&T/SPIE 1905*, 861–870. — the dataset used throughout